PlussMa Distribution Documentation

PlussMa Contributors

Mar 03, 2026

CONTENTS

1 Quickstart **3**
- 1.1 Include Path . . . 3
- 1.2 Sample In One Line . . . 3
- 1.3 Keep Shape And Dtype . . . 3
- 1.4 Evaluate Densities And CDFs . . . 3
- 1.5 Gaussian Mixture . . . 4
- 1.6 Intended Style . . . 4

2 API Reference **5**
- 2.1 `supports` . . . 5
- 2.2 `sample` . . . 5
- 2.3 `sample_like` . . . 6
- 2.4 `pdf` . . . 6
- 2.5 `logpdf` . . . 7
- 2.6 `cdf` . . . 7
- 2.7 `mixture_normal` . . . 7
- 2.8 `mixture_normal_pdf` . . . 8
- 2.9 `mixture_normal_logpdf` . . . 8
- 2.10 `mixture_normal_cdf` . . . 9

3 Distribution Catalog **11**
- 3.1 `normal` . . . 11
- 3.2 `lognormal` . . . 12
- 3.3 `uniform` . . . 12
- 3.4 `exponential` . . . 13
- 3.5 `bernoulli` . . . 14
- 3.6 `beta` . . . 14
- 3.7 `binomial` . . . 15
- 3.8 `bradford` . . . 16
- 3.9 `burr` . . . 17
- 3.10 `cauchy` . . . 17
- 3.11 `chi` . . . 18
- 3.12 `chisquare` . . . 19
- 3.13 `chisqnoncentral` . . . 19
- 3.14 `cosine` . . . 20
- 3.15 `erlang` . . . 21
- 3.16 `extremevalue` . . . 21
- 3.17 `f` . . . 22
- 3.18 `fnoncentral` . . . 23
- 3.19 `laplace` . . . 23

3.20 logistic . . . 24
3.21 geometric . . . 25
3.22 gamma . . . 25
3.23 gompertz . . . 26
3.24 halfcosine . . . 27
3.25 hyperbolicsecant . . . 27
3.26 hypergeometric . . . 28
3.27 inversegaussian . . . 29
3.28 maxwell . . . 29
3.29 nakagami . . . 30
3.30 negativebinomial . . . 31
3.31 normaltrunc . . . 31
3.32 planck . . . 32
3.33 poisson . . . 33
3.34 quadratic . . . 33
3.35 rademacher . . . 34
3.36 studentst . . . 35
3.37 triangular . . . 35
3.38 pareto . . . 36
3.39 lomax . . . 37
3.40 rayleigh . . . 38
3.41 rice . . . 38
3.42 semicircle . . . 39
3.43 skellam . . . 40
3.44 tukeylambda . . . 40
3.45 ushape . . . 41
3.46 vonmises . . . 42
3.47 wald . . . 42
3.48 weibull . . . 43
3.49 yule . . . 44
3.50 zeta . . . 44
3.51 gig . . . 45
3.52 gh . . . 46
3.53 nig . . . 46

4 Build Outputs **49**
4.1 Install Doc Dependencies . . . 49
4.2 Build HTML . . . 49
4.3 Build PDF . . . 49
4.4 Current Outputs . . . 50
4.5 Relevant Code Artifacts . . . 50
4.6 Validation Path . . . 50

The legacy Matlab `randraw.m` surface has been retired. Probability distributions now live in the C++ API as `pluss::distr`, available directly from:

```
#include <pluss/pluss.h>
```

This manual is organized around how users actually consume the layer:

- start with `quickstart` for the short-path workflow
- use `api_reference` when you need the function surface
- use `distribution_catalog` when you need names, aliases, and parameter conventions
- use `build_outputs` when you want to build the docs or the example binary locally

CHAPTER

ONE

QUICKSTART

1.1 Include Path

```
#include <pluss/pluss.h>
```

The distribution layer is exposed as `pluss::distr`. No extra include is needed when PlussMa is part of the build.

1.2 Sample In One Line

```
auto x = pluss::distr::sample("normal", {0.0, 1.0}, {1024});
auto y = pluss::distr::sample("arcsin", {1024});
auto z = pluss::distr::sample("vonmises", {0.0, 3.0}, {1024});
```

1.3 Keep Shape And Dtype

```
auto like = pluss::zeros({2, 4},
    pluss::Config{}
        .with_dtype(pluss::ScalarType::Float32)
        .with_platform(pluss::pCPU)
        .with_pinned(true));

auto gamma_like = pluss::distr::sample_like("gamma", {2.0}, like);
auto signs_like = pluss::distr::sample_like("rademacher", like);
```

`sample_like(...)` preserves the visible tensor contract from `like`: shape, dtype, and platform. It also forwards the pinned flag into the sampling config, but the current runtime does not surface pinned-memory state through `is_pinned()` yet.

1.4 Evaluate Densities And CDFs

```
auto grid = pluss::make_series(std::vector<double>{-2.0, -1.0, 0.0, 1.0, 2.0}, {5});

auto normal_pdf = pluss::distr::pdf("normal", grid, {0.0, 1.0});
auto t_cdf = pluss::distr::cdf("t", grid, {5.0});
auto nig_logpdf = pluss::distr::logpdf("nig", grid, {2.0, 0.5, 0.0, 1.0});
```

1.5 Gaussian Mixture

```
auto weights = pluss::make_series(std::vector<double>{0.65, 0.35}, {2});
auto means = pluss::make_series(std::vector<double>{-1.0, 2.0}, {2});
auto stddevs = pluss::make_series(std::vector<double>{0.5, 1.25}, {2});

auto mix = pluss::distr::mixture_normal(
    weights,
    means,
    stddevs,
    {4096},
    pluss::Config{}.with_dtype(pluss::ScalarType::Float64),
    23ULL,
    true);
```

The last argument switches the base normal generator from pseudo-random draws to Sobol-driven Gaussian draws.

1.6 Intended Style

The user-facing model is:

- keep everything under `pluss::distr`
- pass the distribution name plus a short parameter list
- let `sample_like` carry shape and dtype when you already have a target tensor
- stay on `Series` throughout instead of bouncing between helper wrappers

CHAPTER

TWO

API REFERENCE

2.1 supports

2.1.1 Signature

```cpp
bool pluss::distr::supports(std::string_view name) noexcept;
```

2.1.2 Notes

Checks whether the runtime resolver recognizes a family name or alias before you call `sample`, `pdf`, `logpdf`, or `cdf`.

2.1.3 Examples

```cpp
std::cout << pluss::distr::supports("gauss") << "\n";
std::cout << pluss::distr::supports("zipf") << "\n";
std::cout << pluss::distr::supports("not-a-dist") << "\n";
```

2.2 sample

2.2.1 Signature

```cpp
pluss::Series pluss::distr::sample(
    std::string_view name,
    std::span<const double> params,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0);
```

2.2.2 Notes

Samples a new `Series` from the chosen family. Use the parameter list required by that distribution, pass the output shape through `sizes`, and use `cfg` when you need to control dtype or platform.

2.2.3 Examples

```cpp
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
```

(continues on next page)

(continued from previous page)

```
auto draws = pluss::distr::sample("normal", {0.0, 1.0}, {2, 3}, cfg, 20260301ULL);
```

2.3 sample_like

2.3.1 Signature

```
pluss::Series pluss::distr::sample_like(
    std::string_view name,
    std::span<const double> params,
    const pluss::Series& like,
    std::uint64_t seed = 0);
```

2.3.2 Notes

Uses the same distribution parameters as `sample(...)`, but takes shape, dtype, and platform from `like`. This is the right helper when another `Series` already defines the tensor contract you want.

2.3.3 Examples

```
auto like = pluss::zeros(
    {2, 4},
    pluss::Config{}
        .with_dtype(pluss::ScalarType::Float32)
        .with_platform(pluss::pCPU));

auto out = pluss::distr::sample_like("gamma", {2.0}, like, 77ULL);
```

2.4 pdf

2.4.1 Signature

```
pluss::Series pluss::distr::pdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.4.2 Notes

Evaluates the probability density function for continuous families or the mass function for discrete families at every point in `x`.

2.4.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto dens = pluss::distr::pdf("normal", x, {0.0, 1.0});
```

2.5 logpdf

2.5.1 Signature

```
pluss::Series pluss::distr::logpdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.5.2 Notes

Evaluates the natural log of the density or mass function. Use it when you are building log-likelihood objectives or want to avoid manually taking `log(pdf(...))`.

2.5.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto logdens = pluss::distr::logpdf("nig", x, {2.0, 0.5, 0.0, 1.0});
```

2.6 cdf

2.6.1 Signature

```
pluss::Series pluss::distr::cdf(
    std::string_view name,
    const pluss::Series& x,
    std::span<const double> params = {});
```

2.6.2 Notes

Evaluates the cumulative distribution function at every point in `x`. Use it when your downstream logic is expressed in tail probabilities or threshold probabilities rather than densities.

2.6.3 Examples

```
auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto dist = pluss::distr::cdf("t", x, {5.0});
```

2.7 mixture_normal

2.7.1 Signature

```
pluss::Series pluss::distr::mixture_normal(
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs,
    std::span<const std::int64_t> sizes,
    pluss::Config cfg = {},
    std::uint64_t seed = 0,
    bool use_sobol = false);
```

2.7.2 Notes

Samples from a Gaussian mixture defined by component weights, means, and standard deviations. Set `use_sobol=true` when you want Sobol-driven quasi-random normal draws instead of pseudo-random draws.

2.7.3 Examples

```
auto cfg = pluss::Config{}.with_dtype(pluss::ScalarType::Float64).with_
↪platform(pluss::pCPU);
auto weights = pluss::make_series({0.65, 0.35}, cfg);
auto means = pluss::make_series({-1.0, 2.0}, cfg);
auto stddevs = pluss::make_series({0.5, 1.25}, cfg);

auto mix = pluss::distr::mixture_normal(weights, means, stddevs, {8}, cfg, 101ULL, true);
```

2.8 mixture_normal_pdf

2.8.1 Signature

```
pluss::Series pluss::distr::mixture_normal_pdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.8.2 Notes

Evaluates the density of a Gaussian mixture that you have already parameterized as component `Series` values.

2.8.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto dens = pluss::distr::mixture_normal_pdf(x, weights, means, stddevs);
```

2.9 mixture_normal_logpdf

2.9.1 Signature

```
pluss::Series pluss::distr::mixture_normal_logpdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.9.2 Notes

Evaluates the log-density of the same Gaussian mixture parameterization used by `mixture_normal_pdf(...)`.

2.9.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto logdens = pluss::distr::mixture_normal_logpdf(x, weights, means, stddevs);
```

2.10 mixture_normal_cdf

2.10.1 Signature

```
pluss::Series pluss::distr::mixture_normal_cdf(
    const pluss::Series& x,
    const pluss::Series& weights,
    const pluss::Series& means,
    const pluss::Series& stddevs);
```

2.10.2 Notes

Evaluates the mixture CDF using the same component parameterization as the other Gaussian-mixture helpers.

2.10.3 Examples

```
auto x = pluss::make_series({-2.0, -0.5, 1.0, 3.0}).to(pluss::ScalarType::Float64);
auto weights = pluss::make_series({0.65, 0.35}).to(pluss::ScalarType::Float64);
auto means = pluss::make_series({-1.0, 2.0}).to(pluss::ScalarType::Float64);
auto stddevs = pluss::make_series({0.5, 1.25}).to(pluss::ScalarType::Float64);

auto dist = pluss::distr::mixture_normal_cdf(x, weights, means, stddevs);
```

CHAPTER

THREE

DISTRIBUTION CATALOG

Each section below documents one supported family with the same reference layout used throughout the docs: `Signature`, `Notes`, and `Examples`.

3.1 normal

3.1.1 Signature

```
pluss::distr::sample("normal", {mu, sigma}, sizes, cfg, seed);
pluss::distr::sample("normal", {}, sizes, cfg, seed);
pluss::distr::sample_like("normal", {mu, sigma}, like, seed);
pluss::distr::sample_like("normal", {}, like, seed);
pluss::distr::pdf("normal", x, {mu, sigma});
pluss::distr::pdf("normal", x, {});
pluss::distr::logpdf("normal", x, {mu, sigma});
pluss::distr::logpdf("normal", x, {});
pluss::distr::cdf("normal", x, {mu, sigma});
pluss::distr::cdf("normal", x, {});
```

3.1.2 Notes

Gaussian family. Use the empty parameter list for the standard normal, or pass mean and standard deviation for a shifted/scaled normal.

Aliases: `norm`, `gaussian`, `gauss`.

Parameters: `{mu, sigma}` or `{}` for standard normal.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.1.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("normal", {0.0, 1.0}, {6}, cfg, 11ULL);
auto like_draws = pluss::distr::sample_like("normal", {0.0, 1.0}, like, 11ULL);
```

(continues on next page)

(continued from previous page)

```
auto dens = pluss::distr::pdf("normal", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("normal", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("normal", x, {0.0, 1.0});
```

3.2 lognormal

3.2.1 Signature

```
pluss::distr::sample("lognormal", {mu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("lognormal", {mu, sigma}, like, seed);
pluss::distr::pdf("lognormal", x, {mu, sigma});
pluss::distr::logpdf("lognormal", x, {mu, sigma});
pluss::distr::cdf("lognormal", x, {mu, sigma});
```

3.2.2 Notes

Samples and densities are computed in log-space, then mapped back to the positive real line.

Aliases: `lognorm, cobbdouglas, antilognormal`.

Parameters: `{mu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.2.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("lognorm", {0.0, 0.5}, {6}, cfg, 12ULL);
auto like_draws = pluss::distr::sample_like("lognorm", {0.0, 0.5}, like, 12ULL);
auto dens = pluss::distr::pdf("lognorm", x, {0.0, 0.5});
auto logprob = pluss::distr::logpdf("lognorm", x, {0.0, 0.5});
auto dist = pluss::distr::cdf("lognorm", x, {0.0, 0.5});
```

3.3 uniform

3.3.1 Signature

```
pluss::distr::sample("uniform", {a, b}, sizes, cfg, seed);
pluss::distr::sample_like("uniform", {a, b}, like, seed);
pluss::distr::pdf("uniform", x, {a, b});
pluss::distr::logpdf("uniform", x, {a, b});
pluss::distr::cdf("uniform", x, {a, b});
```

3.3.2 Notes

Draws uniformly on the closed interval defined by the lower and upper bounds.

Aliases: `unif`.

Parameters: `{a, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.3.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.25, 0.5, 0.75, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("uniform", {0.0, 1.0}, {6}, cfg, 13ULL);
auto like_draws = pluss::distr::sample_like("uniform", {0.0, 1.0}, like, 13ULL);
auto dens = pluss::distr::pdf("uniform", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("uniform", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("uniform", x, {0.0, 1.0});
```

3.4 exponential

3.4.1 Signature

```
pluss::distr::sample("exponential", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("exponential", {lambda}, like, seed);
pluss::distr::pdf("exponential", x, {lambda});
pluss::distr::logpdf("exponential", x, {lambda});
pluss::distr::cdf("exponential", x, {lambda});
```

3.4.2 Notes

Uses the rate parameter lambda. Larger lambda concentrates more mass near zero.

Aliases: `exp`.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.4.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.5, 1.0, 1.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("exp", {1.5}, {6}, cfg, 14ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("exp", {1.5}, like, 14ULL);
auto dens = pluss::distr::pdf("exp", x, {1.5});
auto logprob = pluss::distr::logpdf("exp", x, {1.5});
auto dist = pluss::distr::cdf("exp", x, {1.5});
```

3.5 bernoulli

3.5.1 Signature

```
pluss::distr::sample("bernoulli", {p}, sizes, cfg, seed);
pluss::distr::sample_like("bernoulli", {p}, like, seed);
pluss::distr::pdf("bernoulli", x, {p});
pluss::distr::logpdf("bernoulli", x, {p});
pluss::distr::cdf("bernoulli", x, {p});
```

3.5.2 Notes

Discrete 0/1 family. `sample(...)` naturally produces a boolean Series unless you override dtype.

Aliases: `bern`.

Parameters: `{p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.5.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("bern", {0.35}, {8}, cfg, 15ULL);
auto like_draws = pluss::distr::sample_like("bern", {0.35}, like, 15ULL);
auto pmf = pluss::distr::pdf("bern", x, {0.35});
auto logprob = pluss::distr::logpdf("bern", x, {0.35});
auto dist = pluss::distr::cdf("bern", x, {0.35});
```

3.6 beta

3.6.1 Signature

```
pluss::distr::sample("beta", {a, b}, sizes, cfg, seed);
pluss::distr::sample("beta", {m, n, a, b}, sizes, cfg, seed);
pluss::distr::sample_like("beta", {a, b}, like, seed);
pluss::distr::sample_like("beta", {m, n, a, b}, like, seed);
pluss::distr::pdf("beta", x, {a, b});
pluss::distr::pdf("beta", x, {m, n, a, b});
```

(continues on next page)

(continued from previous page)

```
pluss::distr::logpdf("beta", x, {a, b});
pluss::distr::logpdf("beta", x, {m, n, a, b});
pluss::distr::cdf("beta", x, {a, b});
pluss::distr::cdf("beta", x, {m, n, a, b});
```

3.6.2 Notes

Use the two-parameter form for the standard beta on [0,1]. The four-parameter form exposes the shifted/scaled legacy convention.

Aliases: `powerfunction, powerfunc`.

Parameters: `{a, b} or {m, n, a, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.6.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.25, 0.5, 0.75}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("beta", {2.0, 5.0}, {6}, cfg, 16ULL);
auto like_draws = pluss::distr::sample_like("beta", {2.0, 5.0}, like, 16ULL);
auto dens = pluss::distr::pdf("beta", x, {2.0, 5.0});
auto logprob = pluss::distr::logpdf("beta", x, {2.0, 5.0});
auto dist = pluss::distr::cdf("beta", x, {2.0, 5.0});
```

3.7 binomial

3.7.1 Signature

```
pluss::distr::sample("binomial", {n, p}, sizes, cfg, seed);
pluss::distr::sample_like("binomial", {n, p}, like, seed);
pluss::distr::pdf("binomial", x, {n, p});
pluss::distr::logpdf("binomial", x, {n, p});
pluss::distr::cdf("binomial", x, {n, p});
```

3.7.2 Notes

Counts successes across n Bernoulli trials.

Aliases: `binom`.

Parameters: `{n, p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.7.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("binom", {10.0, 0.3}, {6}, cfg, 17ULL);
auto like_draws = pluss::distr::sample_like("binom", {10.0, 0.3}, like, 17ULL);
auto pmf = pluss::distr::pdf("binom", x, {10.0, 0.3});
auto logprob = pluss::distr::logpdf("binom", x, {10.0, 0.3});
auto dist = pluss::distr::cdf("binom", x, {10.0, 0.3});
```

3.8 bradford

3.8.1 Signature

```
pluss::distr::sample("bradford", {c}, sizes, cfg, seed);
pluss::distr::sample_like("bradford", {c}, like, seed);
pluss::distr::pdf("bradford", x, {c});
pluss::distr::logpdf("bradford", x, {c});
pluss::distr::cdf("bradford", x, {c});
```

3.8.2 Notes

Bounded skewed family controlled by the shape parameter `c`.

Aliases: none.

Parameters: `{c}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.8.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.1, 0.4, 0.8}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("bradford", {1.5}, {6}, cfg, 18ULL);
auto like_draws = pluss::distr::sample_like("bradford", {1.5}, like, 18ULL);
auto dens = pluss::distr::pdf("bradford", x, {1.5});
auto logprob = pluss::distr::logpdf("bradford", x, {1.5});
auto dist = pluss::distr::cdf("bradford", x, {1.5});
```

3.9 burr

3.9.1 Signature

```
pluss::distr::sample("burr", {c, k}, sizes, cfg, seed);
pluss::distr::sample_like("burr", {c, k}, like, seed);
pluss::distr::pdf("burr", x, {c, k});
pluss::distr::logpdf("burr", x, {c, k});
pluss::distr::cdf("burr", x, {c, k});
```

3.9.2 Notes

Heavy-tailed positive family. Both parameters control tail shape.

Aliases: `fisk`.

Parameters: `{c, k}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.9.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("burr", {2.0, 3.0}, {6}, cfg, 19ULL);
auto like_draws = pluss::distr::sample_like("burr", {2.0, 3.0}, like, 19ULL);
auto dens = pluss::distr::pdf("burr", x, {2.0, 3.0});
auto logprob = pluss::distr::logpdf("burr", x, {2.0, 3.0});
auto dist = pluss::distr::cdf("burr", x, {2.0, 3.0});
```

3.10 cauchy

3.10.1 Signature

```
pluss::distr::sample("cauchy", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("cauchy", {location, scale}, like, seed);
pluss::distr::pdf("cauchy", x, {location, scale});
pluss::distr::logpdf("cauchy", x, {location, scale});
pluss::distr::cdf("cauchy", x, {location, scale});
```

3.10.2 Notes

Heavy-tailed location-scale family with undefined mean/variance in the classical sense.

Aliases: `lorentz, caushy`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.10.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("cauchy", {0.0, 1.0}, {6}, cfg, 20ULL);
auto like_draws = pluss::distr::sample_like("cauchy", {0.0, 1.0}, like, 20ULL);
auto dens = pluss::distr::pdf("cauchy", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("cauchy", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("cauchy", x, {0.0, 1.0});
```

3.11 chi

3.11.1 Signature

```
pluss::distr::sample("chi", {nu}, sizes, cfg, seed);
pluss::distr::sample("chi", {a, b, nu}, sizes, cfg, seed);
pluss::distr::sample_like("chi", {nu}, like, seed);
pluss::distr::sample_like("chi", {a, b, nu}, like, seed);
pluss::distr::pdf("chi", x, {nu});
pluss::distr::pdf("chi", x, {a, b, nu});
pluss::distr::logpdf("chi", x, {nu});
pluss::distr::logpdf("chi", x, {a, b, nu});
pluss::distr::cdf("chi", x, {nu});
pluss::distr::cdf("chi", x, {a, b, nu});
```

3.11.2 Notes

Chi family. The short form is the standard chi distribution; the longer legacy form exposes location/scale/shape.

Aliases: none.

Parameters: `{nu}` or `{a, b, nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.11.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chi", {3.0}, {6}, cfg, 21ULL);
auto like_draws = pluss::distr::sample_like("chi", {3.0}, like, 21ULL);
auto dens = pluss::distr::pdf("chi", x, {3.0});
auto logprob = pluss::distr::logpdf("chi", x, {3.0});
auto dist = pluss::distr::cdf("chi", x, {3.0});
```

3.12 chisquare

3.12.1 Signature

```
pluss::distr::sample("chisquare", {nu}, sizes, cfg, seed);
pluss::distr::sample_like("chisquare", {nu}, like, seed);
pluss::distr::pdf("chisquare", x, {nu});
pluss::distr::logpdf("chisquare", x, {nu});
pluss::distr::cdf("chisquare", x, {nu});
```

3.12.2 Notes

Standard chi-square family parameterized by degrees of freedom.

Aliases: `chisq, chi2`.

Parameters: `{nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.12.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0, 4.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chisq", {4.0}, {6}, cfg, 22ULL);
auto like_draws = pluss::distr::sample_like("chisq", {4.0}, like, 22ULL);
auto dens = pluss::distr::pdf("chisq", x, {4.0});
auto logprob = pluss::distr::logpdf("chisq", x, {4.0});
auto dist = pluss::distr::cdf("chisq", x, {4.0});
```

3.13 chisqnoncentral

3.13.1 Signature

```
pluss::distr::sample("chisqnoncentral", {lambda, nu}, sizes, cfg, seed);
pluss::distr::sample_like("chisqnoncentral", {lambda, nu}, like, seed);
pluss::distr::pdf("chisqnoncentral", x, {lambda, nu});
pluss::distr::logpdf("chisqnoncentral", x, {lambda, nu});
pluss::distr::cdf("chisqnoncentral", x, {lambda, nu});
```

3.13.2 Notes

Non-central chi-square family with non-centrality `lambda` and degrees of freedom `nu`.

Aliases: `chisqnc, chisqnoncentral, chisqnoncentr, chi2noncentral`.

Parameters: `{lambda, nu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.13.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("chisqnc", {1.5, 4.0}, {6}, cfg, 23ULL);
auto like_draws = pluss::distr::sample_like("chisqnc", {1.5, 4.0}, like, 23ULL);
auto dens = pluss::distr::pdf("chisqnc", x, {1.5, 4.0});
auto logprob = pluss::distr::logpdf("chisqnc", x, {1.5, 4.0});
auto dist = pluss::distr::cdf("chisqnc", x, {1.5, 4.0});
```

3.14 cosine

3.14.1 Signature

```
pluss::distr::sample("cosine", {}, sizes, cfg, seed);
pluss::distr::sample_like("cosine", {}, like, seed);
pluss::distr::pdf("cosine", x, {});
pluss::distr::logpdf("cosine", x, {});
pluss::distr::cdf("cosine", x, {});
```

3.14.2 Notes

Compact-support cosine density centered at zero.

Aliases: none.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.14.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("cosine", {}, {6}, cfg, 24ULL);
auto like_draws = pluss::distr::sample_like("cosine", like, 24ULL);
auto dens = pluss::distr::pdf("cosine", x, {});
auto logprob = pluss::distr::logpdf("cosine", x, {});
auto dist = pluss::distr::cdf("cosine", x, {});
```

3.15 erlang

3.15.1 Signature

```
pluss::distr::sample("erlang", {scale, k}, sizes, cfg, seed);
pluss::distr::sample_like("erlang", {scale, k}, like, seed);
pluss::distr::pdf("erlang", x, {scale, k});
pluss::distr::logpdf("erlang", x, {scale, k});
pluss::distr::cdf("erlang", x, {scale, k});
```

3.15.2 Notes

Integer-shape gamma special case. Useful when you want the classic Erlang parameterization.

Aliases: none.

Parameters: `{scale, k}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.15.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("erlang", {1.25, 3.0}, {6}, cfg, 25ULL);
auto like_draws = pluss::distr::sample_like("erlang", {1.25, 3.0}, like, 25ULL);
auto dens = pluss::distr::pdf("erlang", x, {1.25, 3.0});
auto logprob = pluss::distr::logpdf("erlang", x, {1.25, 3.0});
auto dist = pluss::distr::cdf("erlang", x, {1.25, 3.0});
```

3.16 extremevalue

3.16.1 Signature

```
pluss::distr::sample("extremevalue", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("extremevalue", {location, scale}, like, seed);
pluss::distr::pdf("extremevalue", x, {location, scale});
pluss::distr::logpdf("extremevalue", x, {location, scale});
pluss::distr::cdf("extremevalue", x, {location, scale});
```

3.16.2 Notes

Gumbel-type extreme-value distribution.

Aliases: `extrval, extrvalue, gumbel`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.16.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("extrval", {0.0, 1.0}, {6}, cfg, 26ULL);
auto like_draws = pluss::distr::sample_like("extrval", {0.0, 1.0}, like, 26ULL);
auto dens = pluss::distr::pdf("extrval", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("extrval", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("extrval", x, {0.0, 1.0});
```

3.17 f

3.17.1 Signature

```
pluss::distr::sample("f", {df1, df2}, sizes, cfg, seed);
pluss::distr::sample_like("f", {df1, df2}, like, seed);
pluss::distr::pdf("f", x, {df1, df2});
pluss::distr::logpdf("f", x, {df1, df2});
pluss::distr::cdf("f", x, {df1, df2});
```

3.17.2 Notes

Classical F family with numerator and denominator degrees of freedom.

Aliases: `fdistribution`, `fdistrib`, `fdistr`, `fdist`, `fdis`.

Parameters: `{df1, df2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.17.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("f", {4.0, 6.0}, {6}, cfg, 27ULL);
auto like_draws = pluss::distr::sample_like("f", {4.0, 6.0}, like, 27ULL);
auto dens = pluss::distr::pdf("f", x, {4.0, 6.0});
auto logprob = pluss::distr::logpdf("f", x, {4.0, 6.0});
auto dist = pluss::distr::cdf("f", x, {4.0, 6.0});
```

3.18 fnoncentral

3.18.1 Signature

```
pluss::distr::sample("fnoncentral", {lambda, df1, df2}, sizes, cfg, seed);
pluss::distr::sample_like("fnoncentral", {lambda, df1, df2}, like, seed);
pluss::distr::pdf("fnoncentral", x, {lambda, df1, df2});
pluss::distr::logpdf("fnoncentral", x, {lambda, df1, df2});
pluss::distr::cdf("fnoncentral", x, {lambda, df1, df2});
```

3.18.2 Notes

Non-central F family with non-centrality and two degree-of-freedom parameters.

Aliases: `fnc, fnoncentral, fnoncentr`.

Parameters: `{lambda, df1, df2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.18.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("fnc", {1.5, 4.0, 6.0}, {6}, cfg, 28ULL);
auto like_draws = pluss::distr::sample_like("fnc", {1.5, 4.0, 6.0}, like, 28ULL);
auto dens = pluss::distr::pdf("fnc", x, {1.5, 4.0, 6.0});
auto logprob = pluss::distr::logpdf("fnc", x, {1.5, 4.0, 6.0});
auto dist = pluss::distr::cdf("fnc", x, {1.5, 4.0, 6.0});
```

3.19 laplace

3.19.1 Signature

```
pluss::distr::sample("laplace", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("laplace", {location, scale}, like, seed);
pluss::distr::pdf("laplace", x, {location, scale});
pluss::distr::logpdf("laplace", x, {location, scale});
pluss::distr::cdf("laplace", x, {location, scale});
```

3.19.2 Notes

Two-sided exponential family centered at `location`.

Aliases: `doubleexponential, doubleexp, bilateralexponential, bilateralexp`.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.19.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("laplace", {0.0, 1.0}, {6}, cfg, 29ULL);
auto like_draws = pluss::distr::sample_like("laplace", {0.0, 1.0}, like, 29ULL);
auto dens = pluss::distr::pdf("laplace", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("laplace", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("laplace", x, {0.0, 1.0});
```

3.20 logistic

3.20.1 Signature

```
pluss::distr::sample("logistic", {location, scale}, sizes, cfg, seed);
pluss::distr::sample_like("logistic", {location, scale}, like, seed);
pluss::distr::pdf("logistic", x, {location, scale});
pluss::distr::logpdf("logistic", x, {location, scale});
pluss::distr::cdf("logistic", x, {location, scale});
```

3.20.2 Notes

Logistic location-scale family.

Aliases: none.

Parameters: `{location, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.20.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("logistic", {0.0, 1.0}, {6}, cfg, 30ULL);
auto like_draws = pluss::distr::sample_like("logistic", {0.0, 1.0}, like, 30ULL);
auto dens = pluss::distr::pdf("logistic", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("logistic", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("logistic", x, {0.0, 1.0});
```

3.21 geometric

3.21.1 Signature

```
pluss::distr::sample("geometric", {p}, sizes, cfg, seed);
pluss::distr::sample_like("geometric", {p}, like, seed);
pluss::distr::pdf("geometric", x, {p});
pluss::distr::logpdf("geometric", x, {p});
pluss::distr::cdf("geometric", x, {p});
```

3.21.2 Notes

Counts trials until first success under the runtime convention implemented in `distr.cpp`.

Aliases: `geom`, `furry`.

Parameters: `{p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.21.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("geom", {0.35}, {6}, cfg, 31ULL);
auto like_draws = pluss::distr::sample_like("geom", {0.35}, like, 31ULL);
auto dens = pluss::distr::pdf("geom", x, {0.35});
auto logprob = pluss::distr::logpdf("geom", x, {0.35});
auto dist = pluss::distr::cdf("geom", x, {0.35});
```

3.22 gamma

3.22.1 Signature

```
pluss::distr::sample("gamma", {a}, sizes, cfg, seed);
pluss::distr::sample("gamma", {m, b, a}, sizes, cfg, seed);
pluss::distr::sample_like("gamma", {a}, like, seed);
pluss::distr::sample_like("gamma", {m, b, a}, like, seed);
pluss::distr::pdf("gamma", x, {a});
pluss::distr::pdf("gamma", x, {m, b, a});
pluss::distr::logpdf("gamma", x, {a});
pluss::distr::logpdf("gamma", x, {m, b, a});
pluss::distr::cdf("gamma", x, {a});
pluss::distr::cdf("gamma", x, {m, b, a});
```

3.22.2 Notes

Gamma family. Use the one-parameter shortcut for the standard shape-only form, or the legacy location/scale/shape form when needed.

Aliases: none.

Parameters: `{a} or {m, b, a}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.22.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gamma", {2.5}, {6}, cfg, 32ULL);
auto like_draws = pluss::distr::sample_like("gamma", {2.5}, like, 32ULL);
auto dens = pluss::distr::pdf("gamma", x, {2.5});
auto logprob = pluss::distr::logpdf("gamma", x, {2.5});
auto dist = pluss::distr::cdf("gamma", x, {2.5});
```

3.23 gompertz

3.23.1 Signature

```
pluss::distr::sample("gompertz", {eta, b}, sizes, cfg, seed);
pluss::distr::sample_like("gompertz", {eta, b}, like, seed);
pluss::distr::pdf("gompertz", x, {eta, b});
pluss::distr::logpdf("gompertz", x, {eta, b});
pluss::distr::cdf("gompertz", x, {eta, b});
```

3.23.2 Notes

Positive-support Gompertz family.

Aliases: none.

Parameters: `{eta, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.23.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.25, 0.5, 1.0}).to(pluss::ScalarType::Float64);
```

(continues on next page)

(continued from previous page)

```
auto draws = pluss::distr::sample("gompertz", {1.0, 2.0}, {6}, cfg, 33ULL);
auto like_draws = pluss::distr::sample_like("gompertz", {1.0, 2.0}, like, 33ULL);
auto dens = pluss::distr::pdf("gompertz", x, {1.0, 2.0});
auto logprob = pluss::distr::logpdf("gompertz", x, {1.0, 2.0});
auto dist = pluss::distr::cdf("gompertz", x, {1.0, 2.0});
```

3.24 halfcosine

3.24.1 Signature

```
pluss::distr::sample("halfcosine", {}, sizes, cfg, seed);
pluss::distr::sample_like("halfcosine", {}, like, seed);
pluss::distr::pdf("halfcosine", x, {});
pluss::distr::logpdf("halfcosine", x, {});
pluss::distr::cdf("halfcosine", x, {});
```

3.24.2 Notes

Half-cosine density on bounded support.

Aliases: `hcosine, hcos`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.24.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.25, 0.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hcos", {}, {6}, cfg, 34ULL);
auto like_draws = pluss::distr::sample_like("hcos", like, 34ULL);
auto dens = pluss::distr::pdf("hcos", x, {});
auto logprob = pluss::distr::logpdf("hcos", x, {});
auto dist = pluss::distr::cdf("hcos", x, {});
```

3.25 hyperbolicsecant

3.25.1 Signature

```
pluss::distr::sample("hyperbolicsecant", {}, sizes, cfg, seed);
pluss::distr::sample_like("hyperbolicsecant", {}, like, seed);
pluss::distr::pdf("hyperbolicsecant", x, {});
pluss::distr::logpdf("hyperbolicsecant", x, {});
pluss::distr::cdf("hyperbolicsecant", x, {});
```

3.25.2 Notes

Symmetric hyperbolic-secant family.

Aliases: `hsecant, hsec`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.25.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hsec", {}, {6}, cfg, 35ULL);
auto like_draws = pluss::distr::sample_like("hsec", like, 35ULL);
auto dens = pluss::distr::pdf("hsec", x, {});
auto logprob = pluss::distr::logpdf("hsec", x, {});
auto dist = pluss::distr::cdf("hsec", x, {});
```

3.26 hypergeometric

3.26.1 Signature

```
pluss::distr::sample("hypergeometric", {N, M, n}, sizes, cfg, seed);
pluss::distr::sample_like("hypergeometric", {N, M, n}, like, seed);
pluss::distr::pdf("hypergeometric", x, {N, M, n});
pluss::distr::logpdf("hypergeometric", x, {N, M, n});
pluss::distr::cdf("hypergeometric", x, {N, M, n});
```

3.26.2 Notes

Draws counts without replacement from a finite population.

Aliases: `hypergeom`.

Parameters: `{N, M, n}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.26.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("hypergeom", {20.0, 8.0, 5.0}, {6}, cfg, 36ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("hypergeom", {20.0, 8.0, 5.0}, like, 36ULL);
auto pmf = pluss::distr::pdf("hypergeom", x, {20.0, 8.0, 5.0});
auto logprob = pluss::distr::logpdf("hypergeom", x, {20.0, 8.0, 5.0});
auto dist = pluss::distr::cdf("hypergeom", x, {20.0, 8.0, 5.0});
```

3.27 inversegaussian

3.27.1 Signature

```
pluss::distr::sample("inversegaussian", {mu, lambda}, sizes, cfg, seed);
pluss::distr::sample_like("inversegaussian", {mu, lambda}, like, seed);
pluss::distr::pdf("inversegaussian", x, {mu, lambda});
pluss::distr::logpdf("inversegaussian", x, {mu, lambda});
pluss::distr::cdf("inversegaussian", x, {mu, lambda});
```

3.27.2 Notes

Positive-support inverse Gaussian / Wald-type family with separate mean and shape.

Aliases: `ig, inversegauss, invgauss`.

Parameters: `{mu, lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.27.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("ig", {1.5, 2.0}, {6}, cfg, 37ULL);
auto like_draws = pluss::distr::sample_like("ig", {1.5, 2.0}, like, 37ULL);
auto dens = pluss::distr::pdf("ig", x, {1.5, 2.0});
auto logprob = pluss::distr::logpdf("ig", x, {1.5, 2.0});
auto dist = pluss::distr::cdf("ig", x, {1.5, 2.0});
```

3.28 maxwell

3.28.1 Signature

```
pluss::distr::sample("maxwell", {scale}, sizes, cfg, seed);
pluss::distr::sample_like("maxwell", {scale}, like, seed);
pluss::distr::pdf("maxwell", x, {scale});
pluss::distr::logpdf("maxwell", x, {scale});
pluss::distr::cdf("maxwell", x, {scale});
```

3.28.2 Notes

Maxwell speed distribution.

Aliases: none.

Parameters: `{scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.28.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("maxwell", {1.0}, {6}, cfg, 38ULL);
auto like_draws = pluss::distr::sample_like("maxwell", {1.0}, like, 38ULL);
auto dens = pluss::distr::pdf("maxwell", x, {1.0});
auto logprob = pluss::distr::logpdf("maxwell", x, {1.0});
auto dist = pluss::distr::cdf("maxwell", x, {1.0});
```

3.29 nakagami

3.29.1 Signature

```
pluss::distr::sample("nakagami", {m, omega}, sizes, cfg, seed);
pluss::distr::sample_like("nakagami", {m, omega}, like, seed);
pluss::distr::pdf("nakagami", x, {m, omega});
pluss::distr::logpdf("nakagami", x, {m, omega});
pluss::distr::cdf("nakagami", x, {m, omega});
```

3.29.2 Notes

Positive-support Nakagami family.

Aliases: `nakagamim, nakagami-m`.

Parameters: `{m, omega}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.29.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("nakagami", {1.5, 2.0}, {6}, cfg, 39ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("nakagami", {1.5, 2.0}, like, 39ULL);
auto dens = pluss::distr::pdf("nakagami", x, {1.5, 2.0});
auto logprob = pluss::distr::logpdf("nakagami", x, {1.5, 2.0});
auto dist = pluss::distr::cdf("nakagami", x, {1.5, 2.0});
```

3.30 negativebinomial

3.30.1 Signature

```
pluss::distr::sample("negativebinomial", {r, p}, sizes, cfg, seed);
pluss::distr::sample_like("negativebinomial", {r, p}, like, seed);
pluss::distr::pdf("negativebinomial", x, {r, p});
pluss::distr::logpdf("negativebinomial", x, {r, p});
pluss::distr::cdf("negativebinomial", x, {r, p});
```

3.30.2 Notes

Counts failures before a target number of successes under the implemented convention.

Aliases: `negbinomial, negbinom, pascal`.

Parameters: `{r, p}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.30.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("pascal", {3.0, 0.45}, {6}, cfg, 40ULL);
auto like_draws = pluss::distr::sample_like("pascal", {3.0, 0.45}, like, 40ULL);
auto pmf = pluss::distr::pdf("pascal", x, {3.0, 0.45});
auto logprob = pluss::distr::logpdf("pascal", x, {3.0, 0.45});
auto dist = pluss::distr::cdf("pascal", x, {3.0, 0.45});
```

3.31 normaltrunc

3.31.1 Signature

```
pluss::distr::sample("normaltrunc", {a, b, mu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("normaltrunc", {a, b, mu, sigma}, like, seed);
pluss::distr::pdf("normaltrunc", x, {a, b, mu, sigma});
pluss::distr::logpdf("normaltrunc", x, {a, b, mu, sigma});
pluss::distr::cdf("normaltrunc", x, {a, b, mu, sigma});
```

3.31.2 Notes

Truncated normal clipped to `[a, b]` before normalization.

Aliases: `normaltruncated, gausstrunc`.

Parameters: `{a, b, mu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.31.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("normaltrunc", {-1.0, 1.0, 0.0, 1.0}, {6}, cfg, 41ULL);
auto like_draws = pluss::distr::sample_like("normaltrunc", {-1.0, 1.0, 0.0, 1.0}, like,␣
↪41ULL);
auto dens = pluss::distr::pdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
auto dist = pluss::distr::cdf("normaltrunc", x, {-1.0, 1.0, 0.0, 1.0});
```

3.32 planck

3.32.1 Signature

```
pluss::distr::sample("planck", {lambda, scale}, sizes, cfg, seed);
pluss::distr::sample_like("planck", {lambda, scale}, like, seed);
pluss::distr::pdf("planck", x, {lambda, scale});
pluss::distr::logpdf("planck", x, {lambda, scale});
pluss::distr::cdf("planck", x, {lambda, scale});
```

3.32.2 Notes

Planck-type positive-support family.

Aliases: none.

Parameters: `{lambda, scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.32.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.25, 0.5, 1.0}).to(pluss::ScalarType::Float64);
```

(continues on next page)

(continued from previous page)

```
auto draws = pluss::distr::sample("planck", {2.0, 1.5}, {6}, cfg, 42ULL);
auto like_draws = pluss::distr::sample_like("planck", {2.0, 1.5}, like, 42ULL);
auto dens = pluss::distr::pdf("planck", x, {2.0, 1.5});
auto logprob = pluss::distr::logpdf("planck", x, {2.0, 1.5});
auto dist = pluss::distr::cdf("planck", x, {2.0, 1.5});
```

3.33 poisson

3.33.1 Signature

```
pluss::distr::sample("poisson", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("poisson", {lambda}, like, seed);
pluss::distr::pdf("poisson", x, {lambda});
pluss::distr::logpdf("poisson", x, {lambda});
pluss::distr::cdf("poisson", x, {lambda});
```

3.33.2 Notes

Counts independent events occurring at rate `lambda`.

Aliases: `po`.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.33.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("poisson", {2.0}, {6}, cfg, 43ULL);
auto like_draws = pluss::distr::sample_like("poisson", {2.0}, like, 43ULL);
auto pmf = pluss::distr::pdf("poisson", x, {2.0});
auto logprob = pluss::distr::logpdf("poisson", x, {2.0});
auto dist = pluss::distr::cdf("poisson", x, {2.0});
```

3.34 quadratic

3.34.1 Signature

```
pluss::distr::sample("quadratic", {}, sizes, cfg, seed);
pluss::distr::sample_like("quadratic", {}, like, seed);
pluss::distr::pdf("quadratic", x, {});
pluss::distr::logpdf("quadratic", x, {});
pluss::distr::cdf("quadratic", x, {});
```

3.34.2 Notes

Bounded quadratic family.

Aliases: `quad, quadr`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.34.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-0.5, 0.0, 0.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("quadr", {}, {6}, cfg, 44ULL);
auto like_draws = pluss::distr::sample_like("quadr", like, 44ULL);
auto dens = pluss::distr::pdf("quadr", x, {});
auto logprob = pluss::distr::logpdf("quadr", x, {});
auto dist = pluss::distr::cdf("quadr", x, {});
```

3.35 rademacher

3.35.1 Signature

```
pluss::distr::sample("rademacher", {}, sizes, cfg, seed);
pluss::distr::sample_like("rademacher", {}, like, seed);
pluss::distr::pdf("rademacher", x, {});
pluss::distr::logpdf("rademacher", x, {});
pluss::distr::cdf("rademacher", x, {});
```

3.35.2 Notes

Produces symmetric -1/+1 sign draws.

Aliases: none.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.35.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rademacher", {}, {6}, cfg, 45ULL);
```

(continues on next page)

(continued from previous page)

```
auto like_draws = pluss::distr::sample_like("rademacher", like, 45ULL);
auto pmf = pluss::distr::pdf("rademacher", x, {});
auto logprob = pluss::distr::logpdf("rademacher", x, {});
auto dist = pluss::distr::cdf("rademacher", x, {});
```

3.36 studentst

3.36.1 Signature

```
pluss::distr::sample("studentst", {nu}, sizes, cfg, seed);
pluss::distr::sample("studentst", {nu, chi, eta}, sizes, cfg, seed);
pluss::distr::sample_like("studentst", {nu}, like, seed);
pluss::distr::sample_like("studentst", {nu, chi, eta}, like, seed);
pluss::distr::pdf("studentst", x, {nu});
pluss::distr::pdf("studentst", x, {nu, chi, eta});
pluss::distr::logpdf("studentst", x, {nu});
pluss::distr::logpdf("studentst", x, {nu, chi, eta});
pluss::distr::cdf("studentst", x, {nu});
pluss::distr::cdf("studentst", x, {nu, chi, eta});
```

3.36.2 Notes

Student-t family with optional location/scale extension.

Aliases: `t`.

Parameters: `{nu} or {nu, chi, eta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.36.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("t", {5.0}, {6}, cfg, 46ULL);
auto like_draws = pluss::distr::sample_like("t", {5.0}, like, 46ULL);
auto dens = pluss::distr::pdf("t", x, {5.0});
auto logprob = pluss::distr::logpdf("t", x, {5.0});
auto dist = pluss::distr::cdf("t", x, {5.0});
```

3.37 triangular

3.37.1 Signature

```
pluss::distr::sample("triangular", {a, c, b}, sizes, cfg, seed);
pluss::distr::sample_like("triangular", {a, c, b}, like, seed);
pluss::distr::pdf("triangular", x, {a, c, b});
pluss::distr::logpdf("triangular", x, {a, c, b});
pluss::distr::cdf("triangular", x, {a, c, b});
```

3.37.2 Notes

Triangular family with lower bound `a`, mode `c`, and upper bound `b`.

Aliases: `tri`.

Parameters: `{a, c, b}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.37.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.0, 0.5, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("tri", {0.0, 0.5, 1.0}, {6}, cfg, 47ULL);
auto like_draws = pluss::distr::sample_like("tri", {0.0, 0.5, 1.0}, like, 47ULL);
auto dens = pluss::distr::pdf("tri", x, {0.0, 0.5, 1.0});
auto logprob = pluss::distr::logpdf("tri", x, {0.0, 0.5, 1.0});
auto dist = pluss::distr::cdf("tri", x, {0.0, 0.5, 1.0});
```

3.38 pareto

3.38.1 Signature

```
pluss::distr::sample("pareto", {xm, alpha}, sizes, cfg, seed);
pluss::distr::sample_like("pareto", {xm, alpha}, like, seed);
pluss::distr::pdf("pareto", x, {xm, alpha});
pluss::distr::logpdf("pareto", x, {xm, alpha});
pluss::distr::cdf("pareto", x, {xm, alpha});
```

3.38.2 Notes

Classical Pareto tail family on values at or above the scale threshold.

Aliases: none.

Parameters: `{xm, alpha}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.38.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 4.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("pareto", {1.0, 3.0}, {6}, cfg, 48ULL);
auto like_draws = pluss::distr::sample_like("pareto", {1.0, 3.0}, like, 48ULL);
auto dens = pluss::distr::pdf("pareto", x, {1.0, 3.0});
auto logprob = pluss::distr::logpdf("pareto", x, {1.0, 3.0});
auto dist = pluss::distr::cdf("pareto", x, {1.0, 3.0});
```

3.39 lomax

3.39.1 Signature

```
pluss::distr::sample("lomax", {scale, alpha}, sizes, cfg, seed);
pluss::distr::sample_like("lomax", {scale, alpha}, like, seed);
pluss::distr::pdf("lomax", x, {scale, alpha});
pluss::distr::logpdf("lomax", x, {scale, alpha});
pluss::distr::cdf("lomax", x, {scale, alpha});
```

3.39.2 Notes

Shifted Pareto / Lomax family.

Aliases: `pareto2`.

Parameters: `{scale, alpha}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.39.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("lomax", {2.0, 3.0}, {6}, cfg, 49ULL);
auto like_draws = pluss::distr::sample_like("lomax", {2.0, 3.0}, like, 49ULL);
auto dens = pluss::distr::pdf("lomax", x, {2.0, 3.0});
auto logprob = pluss::distr::logpdf("lomax", x, {2.0, 3.0});
auto dist = pluss::distr::cdf("lomax", x, {2.0, 3.0});
```

3.40 rayleigh

3.40.1 Signature

```
pluss::distr::sample("rayleigh", {scale}, sizes, cfg, seed);
pluss::distr::sample_like("rayleigh", {scale}, like, seed);
pluss::distr::pdf("rayleigh", x, {scale});
pluss::distr::logpdf("rayleigh", x, {scale});
pluss::distr::cdf("rayleigh", x, {scale});
```

3.40.2 Notes

Radial magnitude family derived from isotropic Gaussian components.

Aliases: `rayl`.

Parameters: `{scale}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.40.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rayl", {1.0}, {6}, cfg, 50ULL);
auto like_draws = pluss::distr::sample_like("rayl", {1.0}, like, 50ULL);
auto dens = pluss::distr::pdf("rayl", x, {1.0});
auto logprob = pluss::distr::logpdf("rayl", x, {1.0});
auto dist = pluss::distr::cdf("rayl", x, {1.0});
```

3.41 rice

3.41.1 Signature

```
pluss::distr::sample("rice", {nu, sigma}, sizes, cfg, seed);
pluss::distr::sample_like("rice", {nu, sigma}, like, seed);
pluss::distr::pdf("rice", x, {nu, sigma});
pluss::distr::logpdf("rice", x, {nu, sigma});
pluss::distr::cdf("rice", x, {nu, sigma});
```

3.41.2 Notes

Non-central magnitude family often used in signal models.

Aliases: `rician`.

Parameters: `{nu, sigma}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.41.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.5, 2.5}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("rice", {1.0, 1.0}, {6}, cfg, 51ULL);
auto like_draws = pluss::distr::sample_like("rice", {1.0, 1.0}, like, 51ULL);
auto dens = pluss::distr::pdf("rice", x, {1.0, 1.0});
auto logprob = pluss::distr::logpdf("rice", x, {1.0, 1.0});
auto dist = pluss::distr::cdf("rice", x, {1.0, 1.0});
```

3.42 semicircle

3.42.1 Signature

```
pluss::distr::sample("semicircle", {mu, radius}, sizes, cfg, seed);
pluss::distr::sample_like("semicircle", {mu, radius}, like, seed);
pluss::distr::pdf("semicircle", x, {mu, radius});
pluss::distr::logpdf("semicircle", x, {mu, radius});
pluss::distr::cdf("semicircle", x, {mu, radius});
```

3.42.2 Notes

Compact-support semicircle family.

Aliases: `semicirc, wigner`.

Parameters: `{mu, radius}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.42.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("semicirc", {0.0, 1.0}, {6}, cfg, 52ULL);
auto like_draws = pluss::distr::sample_like("semicirc", {0.0, 1.0}, like, 52ULL);
auto dens = pluss::distr::pdf("semicirc", x, {0.0, 1.0});
auto logprob = pluss::distr::logpdf("semicirc", x, {0.0, 1.0});
auto dist = pluss::distr::cdf("semicirc", x, {0.0, 1.0});
```

3.43 skellam

3.43.1 Signature

```
pluss::distr::sample("skellam", {mu1, mu2}, sizes, cfg, seed);
pluss::distr::sample_like("skellam", {mu1, mu2}, like, seed);
pluss::distr::pdf("skellam", x, {mu1, mu2});
pluss::distr::logpdf("skellam", x, {mu1, mu2});
pluss::distr::cdf("skellam", x, {mu1, mu2});
```

3.43.2 Notes

Difference of two independent Poisson counts.

Aliases: none.

Parameters: `{mu1, mu2}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.43.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("skellam", {2.0, 1.0}, {6}, cfg, 53ULL);
auto like_draws = pluss::distr::sample_like("skellam", {2.0, 1.0}, like, 53ULL);
auto pmf = pluss::distr::pdf("skellam", x, {2.0, 1.0});
auto logprob = pluss::distr::logpdf("skellam", x, {2.0, 1.0});
auto dist = pluss::distr::cdf("skellam", x, {2.0, 1.0});
```

3.44 tukeylambda

3.44.1 Signature

```
pluss::distr::sample("tukeylambda", {lambda}, sizes, cfg, seed);
pluss::distr::sample_like("tukeylambda", {lambda}, like, seed);
pluss::distr::pdf("tukeylambda", x, {lambda});
pluss::distr::logpdf("tukeylambda", x, {lambda});
pluss::distr::cdf("tukeylambda", x, {lambda});
```

3.44.2 Notes

Flexible shape family controlled by the lambda tail parameter.

Aliases: none.

Parameters: `{lambda}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.44.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("tukeylambda", {0.14}, {6}, cfg, 54ULL);
auto like_draws = pluss::distr::sample_like("tukeylambda", {0.14}, like, 54ULL);
auto dens = pluss::distr::pdf("tukeylambda", x, {0.14});
auto logprob = pluss::distr::logpdf("tukeylambda", x, {0.14});
auto dist = pluss::distr::cdf("tukeylambda", x, {0.14});
```

3.45 ushape

3.45.1 Signature

```
pluss::distr::sample("ushape", {}, sizes, cfg, seed);
pluss::distr::sample_like("ushape", {}, like, seed);
pluss::distr::pdf("ushape", x, {});
pluss::distr::logpdf("ushape", x, {});
pluss::distr::cdf("ushape", x, {});
```

3.45.2 Notes

Symmetric U-shaped bounded family.

Aliases: `u`.

Parameters: `{}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.45.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.1, 0.5, 0.9}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("u", {}, {6}, cfg, 55ULL);
auto like_draws = pluss::distr::sample_like("u", like, 55ULL);
auto dens = pluss::distr::pdf("u", x, {});
auto logprob = pluss::distr::logpdf("u", x, {});
auto dist = pluss::distr::cdf("u", x, {});
```

3.46 vonmises

3.46.1 Signature

```
pluss::distr::sample("vonmises", {mu, kappa}, sizes, cfg, seed);
pluss::distr::sample_like("vonmises", {mu, kappa}, like, seed);
pluss::distr::pdf("vonmises", x, {mu, kappa});
pluss::distr::logpdf("vonmises", x, {mu, kappa});
pluss::distr::cdf("vonmises", x, {mu, kappa});
```

3.46.2 Notes

Circular distribution on angles.

Aliases: none.

Parameters: `{mu, kappa}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.46.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("vonmises", {0.0, 2.0}, {6}, cfg, 56ULL);
auto like_draws = pluss::distr::sample_like("vonmises", {0.0, 2.0}, like, 56ULL);
auto dens = pluss::distr::pdf("vonmises", x, {0.0, 2.0});
auto logprob = pluss::distr::logpdf("vonmises", x, {0.0, 2.0});
auto dist = pluss::distr::cdf("vonmises", x, {0.0, 2.0});
```

3.47 wald

3.47.1 Signature

```
pluss::distr::sample("wald", {mu}, sizes, cfg, seed);
pluss::distr::sample_like("wald", {mu}, like, seed);
pluss::distr::pdf("wald", x, {mu});
pluss::distr::logpdf("wald", x, {mu});
pluss::distr::cdf("wald", x, {mu});
```

3.47.2 Notes

Single-parameter Wald special case exposed by the runtime.

Aliases: none.

Parameters: `{mu}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.47.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("wald", {2.0}, {6}, cfg, 57ULL);
auto like_draws = pluss::distr::sample_like("wald", {2.0}, like, 57ULL);
auto dens = pluss::distr::pdf("wald", x, {2.0});
auto logprob = pluss::distr::logpdf("wald", x, {2.0});
auto dist = pluss::distr::cdf("wald", x, {2.0});
```

3.48 weibull

3.48.1 Signature

```
pluss::distr::sample("weibull", {shape, scale}, sizes, cfg, seed);
pluss::distr::sample("weibull", {theta, alpha, beta}, sizes, cfg, seed);
pluss::distr::sample_like("weibull", {shape, scale}, like, seed);
pluss::distr::sample_like("weibull", {theta, alpha, beta}, like, seed);
pluss::distr::pdf("weibull", x, {shape, scale});
pluss::distr::pdf("weibull", x, {theta, alpha, beta});
pluss::distr::logpdf("weibull", x, {shape, scale});
pluss::distr::logpdf("weibull", x, {theta, alpha, beta});
pluss::distr::cdf("weibull", x, {shape, scale});
pluss::distr::cdf("weibull", x, {theta, alpha, beta});
```

3.48.2 Notes

Supports both the short two-parameter Weibull form and the legacy three-parameter location-scale-shape form.

Aliases: `wbl, frechet`.

Parameters: `{shape, scale} or {theta, alpha, beta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.48.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("weibull", {0.0, 2.0, 1.5}, {6}, cfg, 58ULL);
auto like_draws = pluss::distr::sample_like("weibull", {0.0, 2.0, 1.5}, like, 58ULL);
auto dens = pluss::distr::pdf("weibull", x, {0.0, 2.0, 1.5});
auto logprob = pluss::distr::logpdf("weibull", x, {0.0, 2.0, 1.5});
auto dist = pluss::distr::cdf("weibull", x, {0.0, 2.0, 1.5});
```

3.49 yule

3.49.1 Signature

```
pluss::distr::sample("yule", {rho}, sizes, cfg, seed);
pluss::distr::sample_like("yule", {rho}, like, seed);
pluss::distr::pdf("yule", x, {rho});
pluss::distr::logpdf("yule", x, {rho});
pluss::distr::cdf("yule", x, {rho});
```

3.49.2 Notes

Heavy-tailed discrete Yule-Simon family.

Aliases: `yulesimon`.

Parameters: `{rho}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.49.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("yule", {3.0}, {6}, cfg, 59ULL);
auto like_draws = pluss::distr::sample_like("yule", {3.0}, like, 59ULL);
auto pmf = pluss::distr::pdf("yule", x, {3.0});
auto logprob = pluss::distr::logpdf("yule", x, {3.0});
auto dist = pluss::distr::cdf("yule", x, {3.0});
```

3.50 zeta

3.50.1 Signature

```
pluss::distr::sample("zeta", {s}, sizes, cfg, seed);
pluss::distr::sample_like("zeta", {s}, like, seed);
pluss::distr::pdf("zeta", x, {s});
pluss::distr::logpdf("zeta", x, {s});
pluss::distr::cdf("zeta", x, {s});
```

3.50.2 Notes

Zipf/Zeta count family. Use values at or above 1 for the mass function.

Aliases: `zipf`.

Parameters: `{s}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.50.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({1.0, 2.0, 3.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("zipf", {2.5}, {6}, cfg, 60ULL);
auto like_draws = pluss::distr::sample_like("zipf", {2.5}, like, 60ULL);
auto pmf = pluss::distr::pdf("zipf", x, {2.5});
auto logprob = pluss::distr::logpdf("zipf", x, {2.5});
auto dist = pluss::distr::cdf("zipf", x, {2.5});
```

3.51 gig

3.51.1 Signature

```
pluss::distr::sample("gig", {lambda, chi, psi}, sizes, cfg, seed);
pluss::distr::sample_like("gig", {lambda, chi, psi}, like, seed);
pluss::distr::pdf("gig", x, {lambda, chi, psi});
pluss::distr::logpdf("gig", x, {lambda, chi, psi});
pluss::distr::cdf("gig", x, {lambda, chi, psi});
```

3.51.2 Notes

Generalized inverse Gaussian family.

Aliases: none.

Parameters: `{lambda, chi, psi}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.51.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({0.5, 1.0, 2.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gig", {0.5, 1.0, 2.0}, {6}, cfg, 61ULL);
auto like_draws = pluss::distr::sample_like("gig", {0.5, 1.0, 2.0}, like, 61ULL);
auto dens = pluss::distr::pdf("gig", x, {0.5, 1.0, 2.0});
auto logprob = pluss::distr::logpdf("gig", x, {0.5, 1.0, 2.0});
auto dist = pluss::distr::cdf("gig", x, {0.5, 1.0, 2.0});
```

3.52 gh

3.52.1 Signature

```
pluss::distr::sample("gh", {lambda, alpha, beta, mu, delta}, sizes, cfg, seed);
pluss::distr::sample_like("gh", {lambda, alpha, beta, mu, delta}, like, seed);
pluss::distr::pdf("gh", x, {lambda, alpha, beta, mu, delta});
pluss::distr::logpdf("gh", x, {lambda, alpha, beta, mu, delta});
pluss::distr::cdf("gh", x, {lambda, alpha, beta, mu, delta});
```

3.52.2 Notes

Generalized hyperbolic family with explicit skew and tail parameters.

Aliases: none.

Parameters: `{lambda, alpha, beta, mu, delta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.52.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("gh", {1.0, 2.0, 0.5, 0.0, 1.0}, {6}, cfg, 62ULL);
auto like_draws = pluss::distr::sample_like("gh", {1.0, 2.0, 0.5, 0.0, 1.0}, like,␣
↪62ULL);
auto dens = pluss::distr::pdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
auto dist = pluss::distr::cdf("gh", x, {1.0, 2.0, 0.5, 0.0, 1.0});
```

3.53 nig

3.53.1 Signature

```
pluss::distr::sample("nig", {alpha, beta, mu, delta}, sizes, cfg, seed);
pluss::distr::sample_like("nig", {alpha, beta, mu, delta}, like, seed);
pluss::distr::pdf("nig", x, {alpha, beta, mu, delta});
pluss::distr::logpdf("nig", x, {alpha, beta, mu, delta});
pluss::distr::cdf("nig", x, {alpha, beta, mu, delta});
```

3.53.2 Notes

Normal-inverse-Gaussian special case of the generalized hyperbolic family.

Aliases: none.

Parameters: `{alpha, beta, mu, delta}`.

Use `sample_like(...)` when you want the sampled output to inherit shape, dtype, and platform from an existing `Series`.

3.53.3 Examples

```
auto cfg = pluss::Config{}
    .with_dtype(pluss::ScalarType::Float64)
    .with_platform(pluss::pCPU);
auto like = pluss::zeros({2, 4}, cfg.with_dtype(pluss::ScalarType::Float32));

auto x = pluss::make_series({-1.0, 0.0, 1.0}).to(pluss::ScalarType::Float64);
auto draws = pluss::distr::sample("nig", {2.0, 0.5, 0.0, 1.0}, {6}, cfg, 63ULL);
auto like_draws = pluss::distr::sample_like("nig", {2.0, 0.5, 0.0, 1.0}, like, 63ULL);
auto dens = pluss::distr::pdf("nig", x, {2.0, 0.5, 0.0, 1.0});
auto logprob = pluss::distr::logpdf("nig", x, {2.0, 0.5, 0.0, 1.0});
auto dist = pluss::distr::cdf("nig", x, {2.0, 0.5, 0.0, 1.0});
```

CHAPTER

FOUR

BUILD OUTPUTS

4.1 Install Doc Dependencies

```
python3 -m pip install -r requirements.txt
```

4.2 Build HTML

```
make html
```

The HTML output lands in:

```
_build/html
```

Expected entry files:

- `_build/html/index.html`
- `_build/html/quickstart.html`
- `_build/html/api_reference.html`
- `_build/html/distribution_catalog.html`
- `_build/html/build_outputs.html`

4.3 Build PDF

```
make pdf
```

This expects a working LaTeX toolchain with `latexmk`.

Generated files:

- `_build/latex/plussma_distribution_docs.tex`
- `_build/latex/plussma_distribution_docs.pdf`

The final PDF manual is:

```
_build/latex/plussma_distribution_docs.pdf
```

4.4 Current Outputs

The current checked build produced:

- HTML site root: `_build/html/index.html`
- PDF manual: `_build/latex/plussma_distribution_docs.pdf`

4.5 Relevant Code Artifacts

The docs describe the production surface implemented in:

- `PlussMa/Stats/distributions/distr.cpp`
- `PlussMa/include/pluss/subsp/stat/distr_fwd.h`
- `PlussMa/include/pluss/ma/stats.h`
- `PlussMa/examples/Numerical/distributions_demo.cpp`

4.6 Validation Path

The runtime coverage for the documented surface is in:

```
PlussMa/Stats/distributions/test/distr_test.cpp
```